

Leon Hamnett – inventory forecasting assignment

Introduction:

The task was related to developing forecasting models to predict the stock level of the top 5 highest selling items upto 14 days into the future to assist with the problem of intelligent inventory management. The data from a single store for the top 5 items could be used for both training and validation and the data from an additional store for these same 5 items could also be used for testing if desired.

The data provided comprises 3 datasets showing the following properties:

1. sales_train_validation.csv - daily sales from Walmart, spanning the item level up through product categories, departments, and stores across three U.S. regions (California, Texas, and Wisconsin).
2. sell_prices.csv Sell price per item per week per store
3. calendar.csv - Calendar details for the entire time period (for example: day of week, and major events/holidays (Super Bowl, Valentine's Day, Orthodox Easter, etc.))

In total, there are 10 unique stores (CA_1–CA_4, TX_1–TX_3, WI_1–WI_3) and 42,840 individual time series relating to individual products, most of which exhibit intermittent demand patterns (some zero-sales weeks). The time series covers dates from 2011-01-29 to 2016-06-19.

General Methodology:

1. Download raw data
2. Merge 3 datasets across appropriate joining columns, performing transposes and other data manipulation as required.
3. Identifying global top 10 items sold, identify stores which have the most number of global top items in their local-store top 10 most sold items.
4. Choose store with most top-10 global items sold, from this store choose the top 5 best selling items.
5. Subset entire dataset based on store and top 5 items of interest
6. Exploratory analysis – exploring data distributions, trend, seasonality, missing vals etc
7. Feature Engineering – create new features or convert some features to cyclical encoding
8. Preprocess data and split into train and test sets
9. Explore various potential models
10. Select model and train one model per item to make 14-day ahead forecasts for that item
11. Evaluate model forecasting ability per item
12. Tidying code and report write up

Dataset exploration and analysis:

Firstly exploratory data analysis was undertaken to understand the distribution of data within each dataset.

Calendar.csv

The calendar.csv file contains 1,969 weekly records spanning Saturday, January 29, 2011 through Sunday, June 19, 2016, with fourteen columns capturing both temporal and event information. Each row includes the date, a combined week-of-year indicator (wm_yr_wk), the weekday name and its numeric code (wday, where 1 = Saturday), month, year, and a day identifier (d). Two event name/type pairs (event_name_1, event_type_1 and event_name_2, event_type_2) annotate major holidays—162 non-null entries in the first pair and 5 in the second—such as Super Bowl, Valentine’s Day, Easter, Thanksgiving, and Christmas. Finally, three “snap” flags (snap_CA, snap_TX, snap_WI) indicate whether each of the three regions (California, Texas, and Wisconsin) was under a state-level Snap-eligible promotion that week. In total, the dataset catalogs 33 unique events across its holiday columns.

sales_train_validation.csv

The sales_train_validation.csv dataset contains 30,490 rows and 1,919 columns, providing daily unit-sales for every item–store combination over the full training period. The first six columns give the product and location hierarchy—id, item_id, dept_id, cat_id, store_id, and state_id—followed by 1,913 day-indexed sales columns (d_1 through d_1913) that record each a product’s daily unit sales at its specific store. In total, there are 3,049 unique items sold across 10 stores—four in California (CA_1–CA_4), three in Texas (TX_1–TX_3), and three in Wisconsin (WI_1–WI_3).

We first identified the ten most popular items across all stores for the entire period—FOODS_3_090, FOODS_3_586, FOODS_3_252, FOODS_3_555, FOODS_3_714, FOODS_3_587, FOODS_3_694, FOODS_3_226, FOODS_3_202, and FOODS_3_723. Next, we computed each store’s own top-10 best-selling items and counted how many of those matched the global top ten. The overlap by store was as follows: CA_1 (6), CA_2 (4), CA_3 (5), CA_4 (6), TX_1 (9), TX_2 (7), TX_3 (7), WI_1 (5), WI_2 (2), and WI_3 (6). From a business perspective, we selected TX_1—the store with nine global top-10 items in its local top ten—as our primary forecasting pilot, since its sales mix most closely mirrors overall trends and therefore offers the greatest leverage for a broadly applicable demand model. The top items sold from store Tx_11 were as follows: 'FOODS_3_586','FOODS_3_090','FOODS_3_555','FOODS_3_252','FOODS_3_587'.

sell_prices.csv

The pricing dataset contains 6,841,121 records and four fields: store_id and item_id (both as strings), wm_yr_wk (an integer representing the year–week), and sell_price (a floating-point value).

Analysis via merged dataset:

Once the datasets were combined and joined in an appropriate manner, we obtained the merged dataset containing the information for one specific store, and the daily sells of the top 5 products. The raw time series as well as for as rolling mean for each variable of interest was visualised. Also autocorrelation and partial autocorrelation plots were made for each target variable.

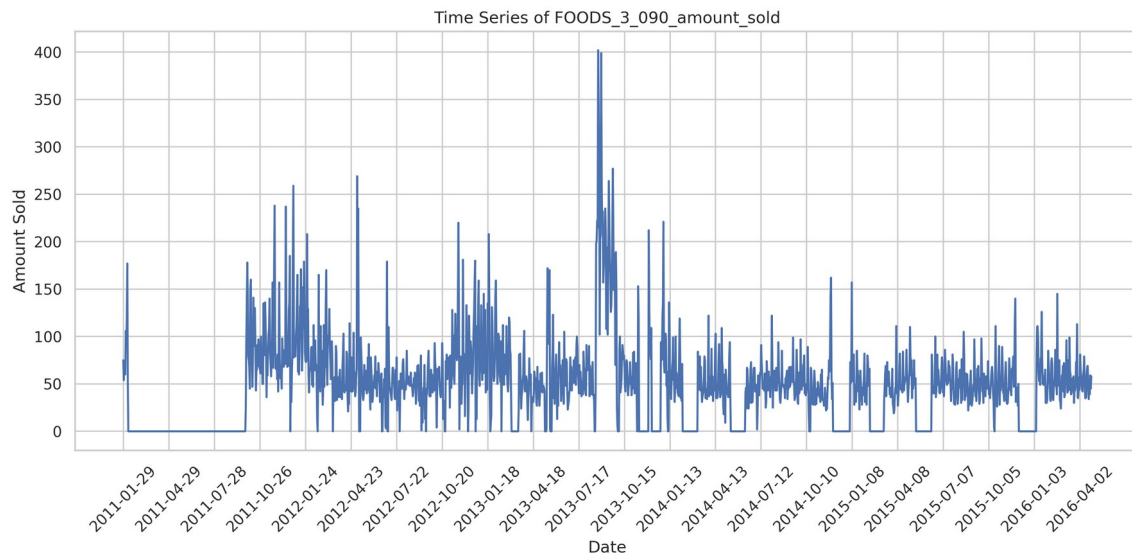


Figure 1: Raw data from Item: Foods_3_090

During this initial investigation, it was realised that for some of the target features of interest, there were large periods where the product registered only zeros. The assumption that was made during the modelling process was that these areas of zero products sold, were due to data entry errors, and that the products did actually sell items on this day. Afterwards each target variable was examined for the presence of null values, no null values were found across all 5 target variables.

After the visual inspection, the investigation moved on to investigating the trend and seasonality present within each target feature. A basic linear regression was undertaken to estimate the trend for each target. Each target feature did not demonstrate large trend values across the entire dataset:

Trend for FOODS_3_586_amount_sold: -0.0128

Trend for FOODS_3_090_amount_sold: 0.0007

Trend for FOODS_3_555_amount_sold: -0.0198

Trend for FOODS_3_252_amount_sold: -0.0030

Trend for FOODS_3_587_amount_sold: 0.0063

Next we examined trend/seasonality/residual plots taking a seasonality assumption of 7 days, 30 days or 365 days for each target of interest. By examining the seasonality plot with the clearest oscillations, we can determine the closest seasonality assumption. For all target features, the strongest seasonality was demonstrated when the period was set to 30 (as shown in the below comparison). As such during the model training, the seasonality was set to 30 periods.

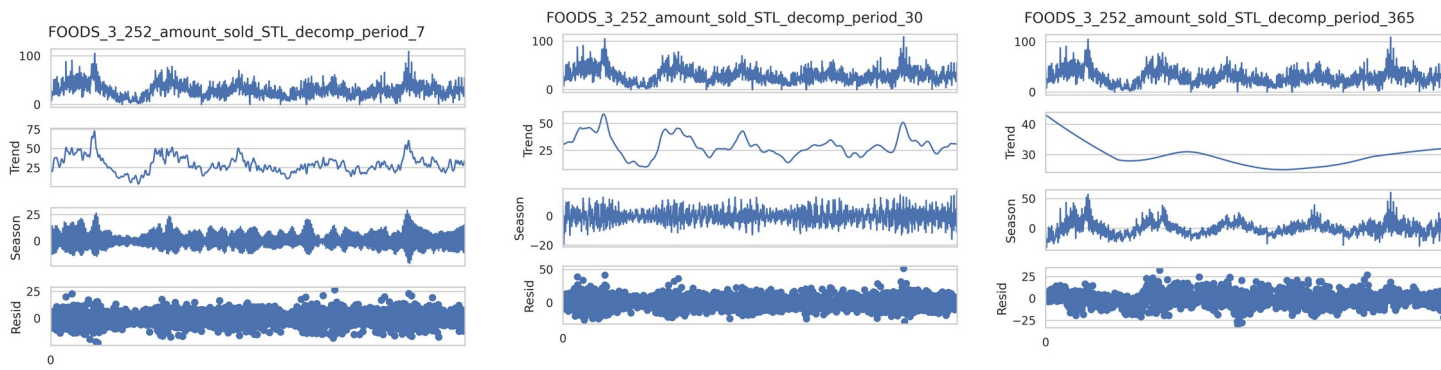


Figure 2: Seasonality and trend plots with assumed period- Left: 7, Middle: 30, Right:365

Feature engineering

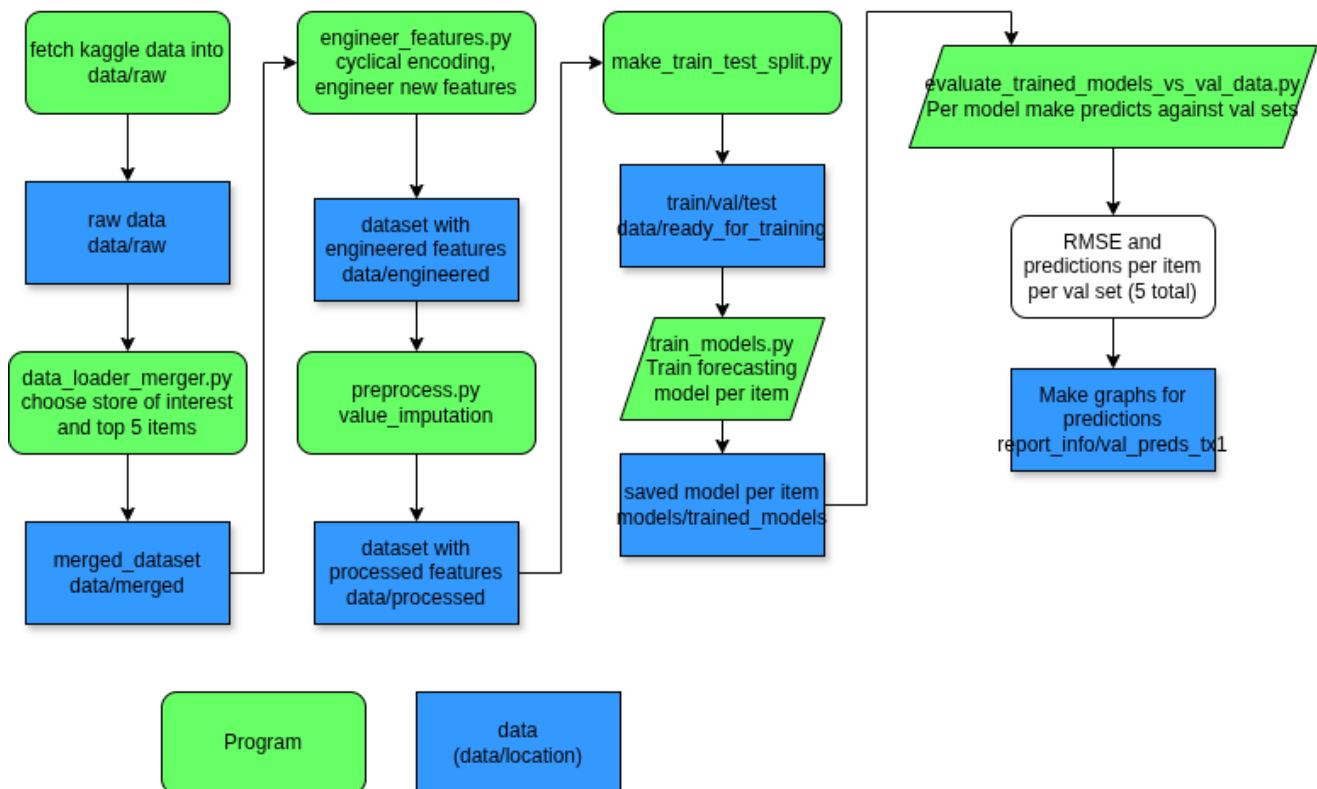
Several time-based features were engineered from the merged sales–calendar–price dataset. We extracted the day of month from the date field, then flagged each row if the row is on a weekend or public holiday (`is_weekend`) and public holiday (`is_public_holiday`) based on non-null event name.

We also computed a numerical measure counting down the days to the next public holiday, assuming that sales for certain goods would spike as holidays were approaching.

Finally, to capture cyclical patterns in the time fields, we transformed the ordinal week-day (`wday`), month, and day-of-month into sine and cosine pairs (e.g. `wday_sin`, `wday_cos`, etc.), preserving their periodic structure for model input and helping the model to learn similarities between min and max values on a cyclical scale eg day 1 of a month is close to day 31 of the previous month

Overall Data/Model flow throughout project:

This section gives a brief summary of the data flow throughout the project and how models are trained and evaluated using this dataflow:



Stage 1: Combining datasets to create merged data:

Raw data to merged data (via data_loader_merger.py)

First we load each CSV into a Pandas DataFrame, filters the sales to `store_id == 'TX_1'` and the five selected `item_ids`, drops unused hierarchy columns, transposes so that each item becomes a column (with rows corresponding to day indices `d_1...d_1913`), and renames the first column to `day_id`. Next, it merges in calendar features (date, week, weekday, month, year, holiday flags, SNAP flags), drops now-redundant columns, and then pivots the sell-price DataFrame to align prices by `wm_yr_wk` for those same five items. Finally, it joins the price data to the sales + calendar table, renames columns to distinguish “`_amount_sold`” and “`_sell_price`”, converts `date` to a datetime index, and writes out the merged result for downstream modeling.

Stage 2: Engineering Features:

Merged data to feature engineered data (via engineer_features.py)

This script takes the previously merged sales–calendar–price dataset for TX_1’s top five items, engineers several time-based features including cycling encoding. After loading the CSV and parsing its `date` column, it extracts the day of month and flags weekends (Saturday/Sunday) and public holidays (any non-null `event_name_1`). It then computes, for each date, the number of days until the next holiday. Finally, it transforms three ordinal date fields—weekday (`wday`), month, and day of month—into four cyclical sine/cosine features each to preserve their periodic nature. The resulting DataFrame, now containing both raw and engineered features, is saved to an “engineered” folder for use in downstream forecasting pipelines.

Stage 3: Preprocessing Data:

Engineer data to preprocessed data (via `preprocess_data.py`)

This script reads in the previously engineered dataset, then applies imputation to each of the five target sales series (`FOODS_3_586_amount_sold`, etc.). Three imputation functions are defined—an STL-based seasonal–trend imputer, a 5th-order polynomial interpolator, and the imputation which was finally used a simple mean-fill that replaces zeros with the series mean. For each output variable we use the imputation to fill values with zeroes.

Stage 4: Making train/test splits:

processed_data to ready_for_training (via `make_train_test_split`)

This script takes the fully processed, imputed dataset for TX_1’s top five items and partitions it into training, validation, and test sets for modeling. It first loads the combined DataFrame (with date parsed), then assigns 90% of the rows to a “train+validation” block and the remaining 10% to a hold-out test set. The train+validation block is further split by reserving the last 14 days as a true validation fold and using everything before that for training. The script reports the resulting row counts for each split and writes out four CSVs—`tx1_train_val.csv`, `tx1_train.csv`, `tx1_val.csv`, and `tx1_test.csv`—into the `data/ready_for_training` directory.

Stage 5: Training models:

Via `train_models.py`

This training script loads the pre-split TX_1 dataset, isolates ten engineered calendar features plus each of the five top-selling item series in turn, and uses PyCaret’s time-series API to fit and save an Random Forest w/ Cond. Deseasonalize & Detrending model and pipeline for generating 14-day forecasts. During experimentations comparing the performance of a large number of forecasting models, this model was chosen as it seemed to show consistently good predictive performance and the lowest root mean square error (RMSE) for each target variable.

During the training process for each target series, it:

- Subsets to the relevant exogenous features plus the target column,
- Initializes a `TSForecastingExperiment` with sliding-window cross-validation and linear target imputation,

- Trains the model on the training set and
- Persists the model artifact under models/trained_models.

Stage 6: Validation of trained models

Via `evaluate_trained_models_vs_val_data.py`

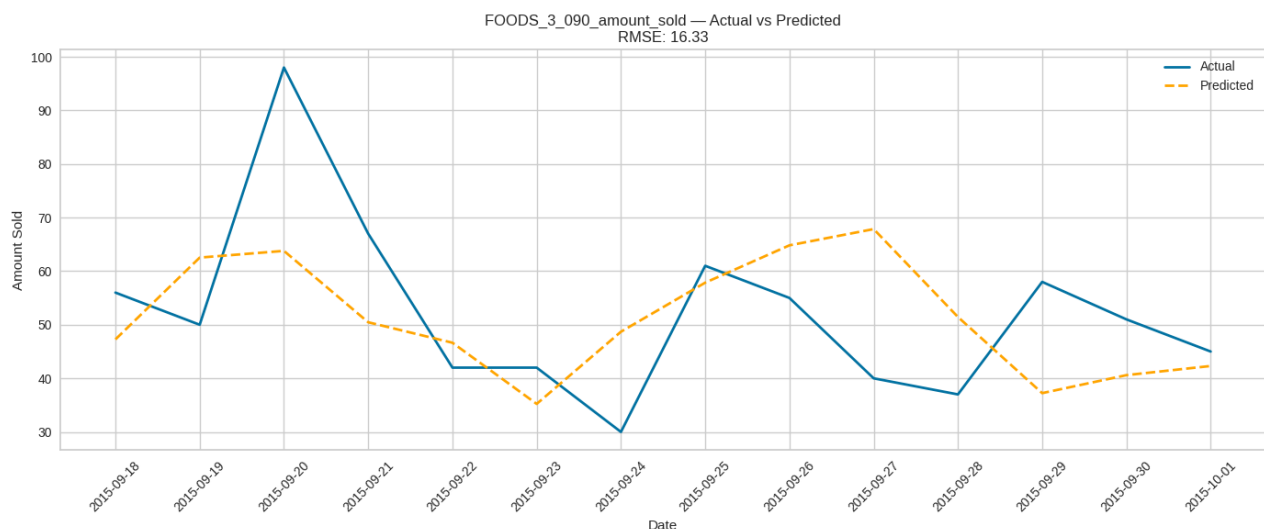
Each trained model for forecasting a specific item is used to generate 14-day forecasts against the hold-out validation window. For each item/model it reloads the pycaret experiment, loads the trained model, calls the predict function to generate predictions for the next 14 days, computes the RMSE between forecasted and actual sales, and then plots actual values vs. predicted values time and saves each chart.

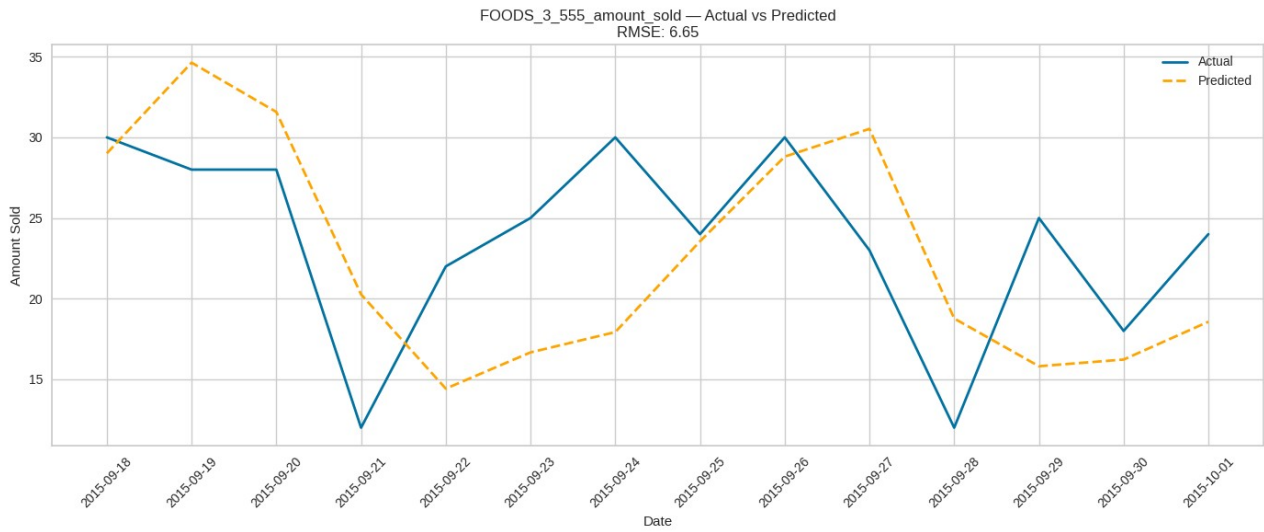
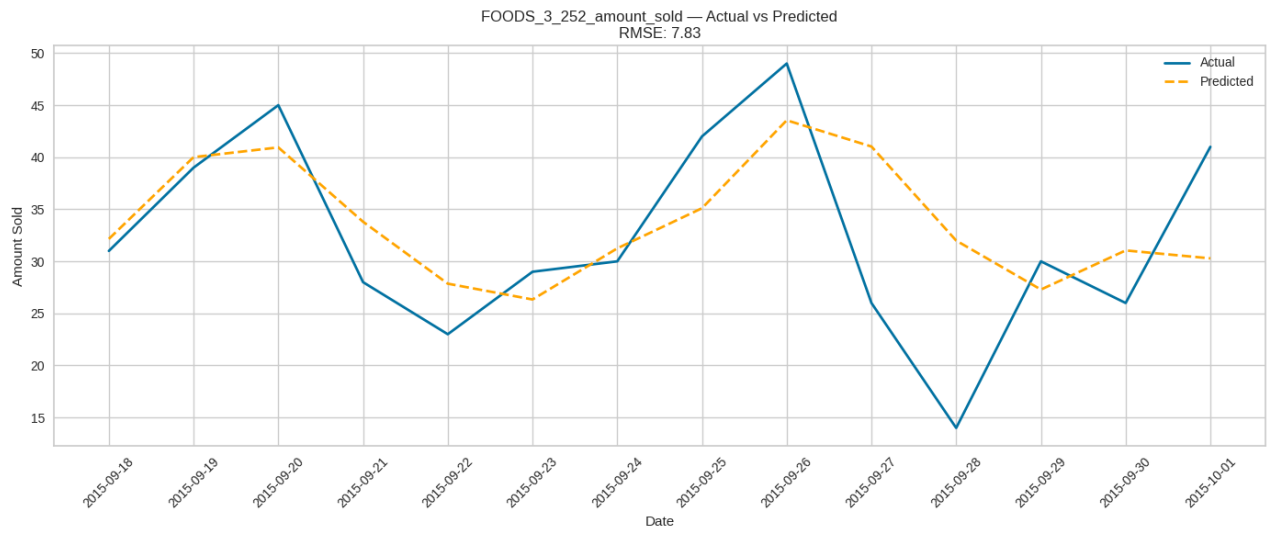
Discussion of modelling results:

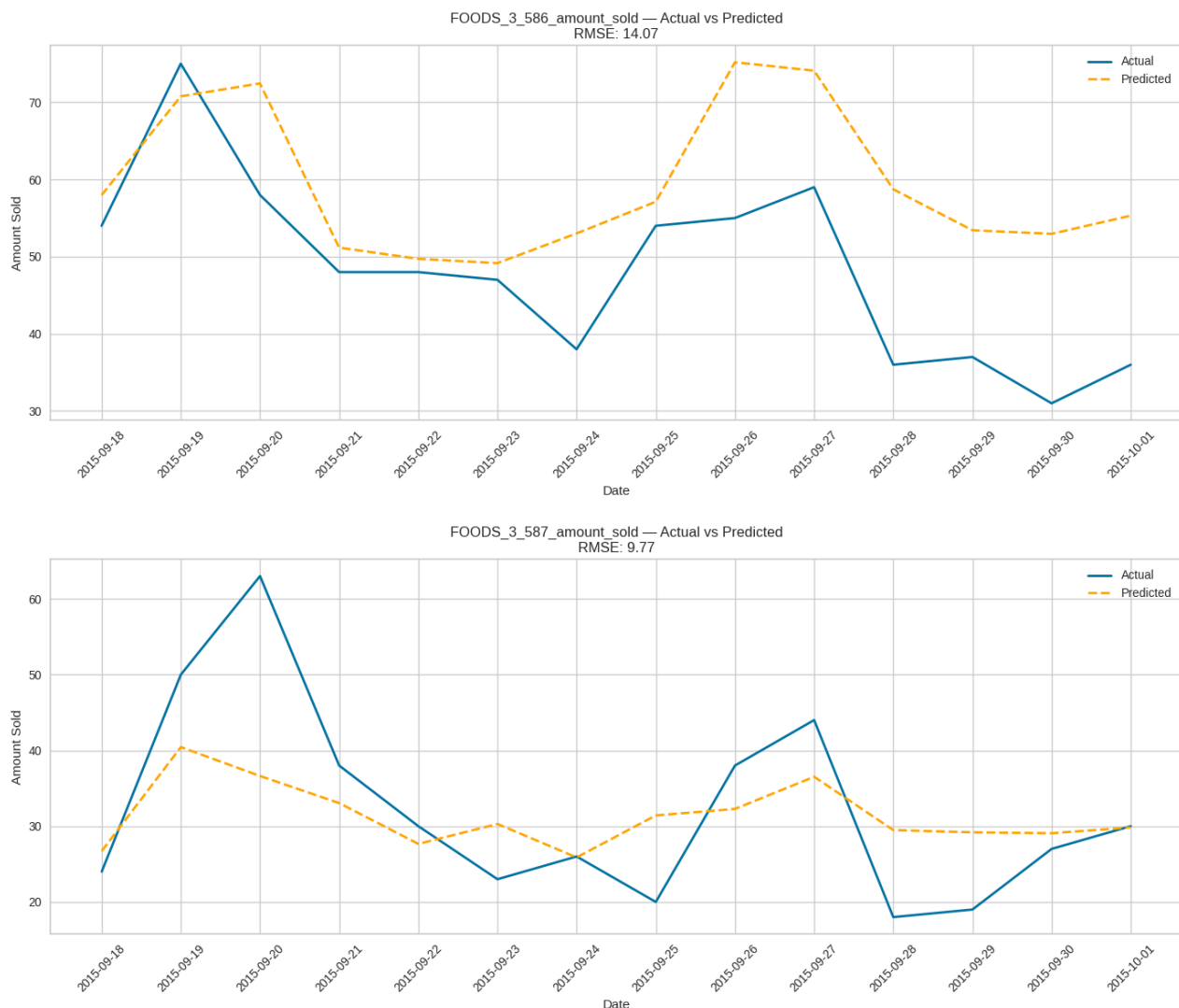
Models evaluated via `evaluate_trained_models_vs_val_data`

Now that we have a model that is trained to forecast the demand for a specific item upto 14 days in the future, we can evaluate each model against the validation data for that same item. During the training process the last 14 data points were not used to evaluate how the model would perform on unseen data.

The results for each specific item/model against the validation data are given below showing actual values in solid blue against the predicted values in dashed orange:







As shown in the above images, we are managing to roughly track future increases or decreases in the actual data, and obtained RMSE values ranging from 8 – 16.

Challenges Faced:

During the project I faced several technical challenges, which could be resolved if I had more time to debug and consider solutions.

- **Imputation Robustness:** My initial linear fills ignored seasonality and trend, and my STL-based residual imputation were not correctly returning reasonable imputations. Given more time, I'd refine the STL pipeline—tune the seasonal period, handle edge cases in residual grouping, and perhaps incorporate a multivariate imputer that borrows strength from related SKUs to fill gaps more accurately.
- **Automated Model Benchmarking:** I wanted to compare a number of models using pycarets automated model comparison features but I hit schema mismatches and struggled with workflow getting the model comparison to complete correctly. To fix this, I'd build a custom sktime-based benchmarking script: programmatic loops over forecasters (ARIMA, ETS, TCN, RF), unified train/validation splits, and saved results in a dataframe—giving me full control over inputs, horizons, and metrics.

- **Rolling-Window Validation:** I wanted true sequential forecasting on a longer validation horizon but ran into prediction constraints in the pycaret package related to the training and validation data. In the end, I had to stick to a validation period of 14 days whereas a longer validation time period may give improved knowledge of a model's performance.
- **Walk forwards prediction:** with more time, I'd implement a walk-forward routine in sktime, for example fitting on an expanding window and forecasts into the future one step at a time. This would mirror real-world deployment. It could also be possible to implement multi-horizon forecasting to forecast multiple steps at once.

Recommendations for Future work:

For future work, I'd start by validating the zero-sales entries. It would be prudent to investigate if these periods of 0s are true "no demand" days or data recording errors? Or perhaps the store ran out of stock. If they're data collection artifacts, we could apply more robust imputation (e.g. refining the STL + residual-imputation routine or using multivariate k-nearest imputation across related products).

In this project, univariate forecasting was used with exogenous features. However we could augment the models with cross-product signal, leveraging item-association patterns so that sales of complimentary products inform forecasts of each other. Additionally, during this project statistical and classical ML models were used, however improved performance may be achieved with the usage of LSTM models or temporal fusion transformers.

If the ultimate goal is inventory optimization, framing the task as a reinforcement-learning problem could be a powerful approach: an agent would learn ordering policies by trading off holding, ordering, and stockout penalties against profit.

On the engineering and pipeline side, automating the full pipeline with MLflow (for data and model versioning), adding unit/integration tests, and building CI/CD alerts would improve reliability and traceability. Finally, once in production, we'd layer in MLOps best practices such as feature stores, model registries, and continuous drift monitoring to detect shifts in input data and demand patterns and trigger retraining automatically. It could also be worthwhile to dockerise the model training and validation environments to ensure

Appendices:

Appendix 1 – Top items sold per store across entire timeline:

Top 10 items for store CA_1:

	store_id	item_id	total_sales
702	CA_1	FOODS_3_090	127203
1198	CA_1	FOODS_3_586	87691
864	CA_1	FOODS_3_252	74971
732	CA_1	FOODS_3_120	61899
1325	CA_1	FOODS_3_714	54080
1199	CA_1	FOODS_3_587	51717
1417	CA_1	FOODS_3_808	46094
411	CA_1	FOODS_2_197	40698
692	CA_1	FOODS_3_080	39145
1167	CA_1	FOODS_3_555	37871

Top 10 items for store CA_2:

	store_id	item_id	total_sales
4247	CA_2	FOODS_3_586	63416
3913	CA_2	FOODS_3_252	57328
3751	CA_2	FOODS_3_090	56169
4216	CA_2	FOODS_3_555	31001
3263	CA_2	FOODS_1_218	28845
3741	CA_2	FOODS_3_080	28703
4466	CA_2	FOODS_3_808	25811
4462	CA_2	FOODS_3_804	25052
4296	CA_2	FOODS_3_635	22368
3949	CA_2	FOODS_3_288	21951

Top 10 items for store CA_3:

	store_id	item_id	total_sales
6800	CA_3	FOODS_3_090	250502
7296	CA_3	FOODS_3_586	134386
6830	CA_3	FOODS_3_120	88795
6962	CA_3	FOODS_3_252	81456
7251	CA_3	FOODS_3_541	80491
7345	CA_3	FOODS_3_635	80222
7515	CA_3	FOODS_3_808	72591
7297	CA_3	FOODS_3_587	70324
7265	CA_3	FOODS_3_555	59092
7391	CA_3	FOODS_3_681	55537

Top 10 items for store CA_4:

	store_id	item_id	total_sales
9849	CA_4	FOODS_3_090	52264
10345	CA_4	FOODS_3_586	32557
10346	CA_4	FOODS_3_587	24333
10011	CA_4	FOODS_3_252	23417
10564	CA_4	FOODS_3_808	21282
10394	CA_4	FOODS_3_635	20895
10472	CA_4	FOODS_3_714	19265
9961	CA_4	FOODS_3_202	18558
10250	CA_4	FOODS_3_491	18533
10300	CA_4	FOODS_3_541	17811

Top 10 items for store TX_1:

	store_id	item_id	total_sales
13394	TX_1	FOODS_3_586	112454
12898	TX_1	FOODS_3_090	93684
13363	TX_1	FOODS_3_555	69516

13060	TX_1	FOODS_3_252	56294
13395	TX_1	FOODS_3_587	48277
13521	TX_1	FOODS_3_714	43567
13010	TX_1	FOODS_3_202	37940
13502	TX_1	FOODS_3_694	36567
13530	TX_1	FOODS_3_723	31907
13415	TX_1	FOODS_3_607	28493

Top 10 items for store TX_2:

	store_id	item_id	total_sales
16443	TX_2	FOODS_3_586	192835
15947	TX_2	FOODS_3_090	119496
16109	TX_2	FOODS_3_252	114153
16412	TX_2	FOODS_3_555	97496
16234	TX_2	FOODS_3_377	63174
16444	TX_2	FOODS_3_587	50127
16570	TX_2	FOODS_3_714	45555
15459	TX_2	FOODS_1_218	44873
16059	TX_2	FOODS_3_202	43772
15888	TX_2	FOODS_3_030	43041

Top 10 items for store TX_3:

	store_id	item_id	total_sales
19492	TX_3	FOODS_3_586	150122
18996	TX_3	FOODS_3_090	114854
19158	TX_3	FOODS_3_252	86632
19461	TX_3	FOODS_3_555	77673
19283	TX_3	FOODS_3_377	73797
18937	TX_3	FOODS_3_030	47976
19108	TX_3	FOODS_3_202	41551

19619	TX_3	FOODS_3_714	40650
19493	TX_3	FOODS_3_587	40306
18868	TX_3	FOODS_2_360	39026

Top 10 items for store WI_1:

	store_id	item_id	total_sales
22181	WI_1	FOODS_3_226	78993
22649	WI_1	FOODS_3_694	41228
22045	WI_1	FOODS_3_090	34852
22541	WI_1	FOODS_3_586	32119
22668	WI_1	FOODS_3_714	30980
22446	WI_1	FOODS_3_491	26462
22623	WI_1	FOODS_3_668	25533
21998	WI_1	FOODS_3_042	25010
22035	WI_1	FOODS_3_080	24765
21426	WI_1	FOODS_1_085	21174

Top 10 items for store WI_2:

	store_id	item_id	total_sales
25230	WI_2	FOODS_3_226	69966
25698	WI_2	FOODS_3_694	59235
25012	WI_2	FOODS_3_007	52020
25238	WI_2	FOODS_3_234	50043
24966	WI_2	FOODS_2_360	48043
25714	WI_2	FOODS_3_711	42212
25380	WI_2	FOODS_3_376	42077
25551	WI_2	FOODS_3_547	41089
25416	WI_2	FOODS_3_412	39266
25466	WI_2	FOODS_3_462	37946

Top 10 items for store WI_3:

	store_id	item_id	total_sales
28143	WI_3	FOODS_3_090	121434
28279	WI_3	FOODS_3_226	98227
28639	WI_3	FOODS_3_586	86080
28747	WI_3	FOODS_3_694	85633
28608	WI_3	FOODS_3_555	57969
28371	WI_3	FOODS_3_318	54770
28766	WI_3	FOODS_3_714	54755
28734	WI_3	FOODS_3_681	48163
28173	WI_3	FOODS_3_120	43376
28287	WI_3	FOODS_3_234	42582

Appendix 2 – Summary statistics showing the distribution of each target of interest

ummary stats:

Summary statistics for FOODS_3_586_amount_sold:

count	1913.000000
mean	58.784109
std	19.713877
min	0.000000
25%	46.000000
50%	56.000000
75%	70.000000
max	162.000000

Name: FOODS_3_586_amount_sold, dtype: float64

Summary statistics for FOODS_3_090_amount_sold:

count	1913.000000
mean	48.972295

std	46.050259
min	0.000000
25%	0.000000
50%	46.000000
75%	67.000000
max	402.000000

Name: FOODS_3_090_amount_sold, dtype: float64

Summary statistics for FOODS_3_555_amount_sold:

count	1913.000000
mean	36.338735
std	18.448596
min	0.000000
25%	23.000000
50%	33.000000
75%	48.000000
max	135.000000

Name: FOODS_3_555_amount_sold, dtype: float64

Summary statistics for FOODS_3_252_amount_sold:

count	1913.000000
mean	29.427078
std	14.550915
min	0.000000
25%	19.000000
50%	28.000000
75%	38.000000
max	109.000000

Name: FOODS_3_252_amount_sold, dtype: float64

Summary statistics for FOODS_3_587_amount_sold:

count	1913.000000
mean	25.236278
std	15.277475
min	0.000000
25%	16.000000
50%	25.000000
75%	35.000000
max	88.000000

Appendix 3: All Seasonal/Trend/Residual graphs (per item/per period of 7/30/365)

